Vehicle Rental SaaS — Feature Functional Blueprint

Feature-by-feature: what it does, where it appears, who uses it, workflow, API responsibility, data and dependencies

Goal: This document is the functional map for the coding agent and product team. Every feature is defined by its purpose, UI location, user, workflow, backend responsibility and data ownership.

1. Product surfaces

• Admin Dashboard — rental company owner, manager and staff.

• Public Booking Portal — customers search, book and pay.

• Customer Account — upcoming bookings, history, profile and documents.

• Background Workers — reminders, notifications, reports, webhooks and scheduled jobs.

• Platform Admin — SaaS-level tenants, plans and usage.

2. Feature map

• Dashboard: KPIs, charts, alerts and quick actions.

• Operations: Bookings, Calendar, Pickup & Return, Active Rentals, Availability.

• Vehicles: Fleet, Categories, Documents, Maintenance, Damage, Locations.

• Customers: Profiles, Verification, Rental History, Blacklist.

• Payments: Transactions, Pending Payments, Deposits, Refunds, Invoices.

• Contracts: Contracts, Templates, Signatures, Expiry tracking.

• Expenses: Expenses, Fuel, Maintenance Costs, Other Costs, Categories.

• Reports: Revenue, Bookings, Fleet Utilization, Customers, Payments, Expenses, P&L.

• Marketing: Coupons, Discounts, Campaigns, Referral Codes.

• Staff: Users, Roles, Permissions, Activity Logs.

• Notifications: Center, Templates, Automation Rules.

• Online Booking: Booking Page, Pricing Rules, Availability Rules, Website Settings.

• Subscription: Plan, Billing, Usage, Invoices.

• Settings: Company, Locations, Hours, Rental Rules, Pricing, Tax, Payment Methods, Integrations, Branding, Security.

3. Feature-by-feature specification

Dashboard

Who: Owner / Manager / Staff

Where: Dashboard landing page

What should show:

• Revenue KPIs; active rentals; today's bookings; available/rented vehicles; pending payments; maintenance count.

• Revenue, booking and fleet-utilization charts.

• Today's pickups/returns, overdue rentals, service due, expiring documents, recent bookings/payments.

• Quick actions: New Booking, Add Vehicle, Add Customer, Record Payment, Start Rental, Complete Return.

What should happen: Load tenant-scoped aggregates. KPI click-throughs open filtered operational pages. Use optimized SQL; do not load the entire database.

```
<b>Backend/API:</b> GET /api/v1/dashboard/summary; GET /api/v1/dashboard/revenue; GET /api/v1/dashboard/bookings; GET /api/v1/dashboard/fleet-utilization
```

Main data: Dashboard is read-only aggregation over bookings, payments, vehicles, maintenance and customers.

Bookings

Who: Owner / Manager / Reservation Staff

Where: Operations > Bookings; Booking detail

What should show:

• Table: booking number, customer, vehicle, pickup/return, location, amount, payment status, booking status.

• Filters: dates, status, customer, vehicle, location, payment status and source.

• Detail: customer, vehicle, pricing, deposit, payments, contract, documents, notes and activity.

• Actions: confirm, edit, cancel, payment, pickup, return and extend.

What should happen: Create: customer → dates/location → vehicle → availability → pricing → discount/tax/deposit → confirmation. Recheck availability on the server during the transaction. Cancellation and extension must follow tenant rules.

```
<b>Backend/API:</b> GET/POST /api/v1/bookings; GET/PATCH /api/v1/bookings/{id}; POST /api/v1/bookings/{id}/confirm; POST /api/v1/bookings/{id}/cancel; POST /api/v1/bookings/{id}/extend
```

Main data: bookings, status history, pricing snapshot, payments, deposits, audit logs.

Calendar

Who: Manager / Staff

Where: Operations > Calendar

What should show:

• Day/week/month views; vehicle/resource view; booking blocks; availability overlay; conflict indicators.

• Click booking → detail. Drag/resize only when permission allows.

What should happen: Calendar is a visualization, never the source of truth. Schedule changes must recheck overlap and commit transactionally.

```
<b>Backend/API:</b> GET /api/v1/calendar; PATCH /api/v1/bookings/{id}/schedule
```

Main data: bookings, vehicles, locations, availability.

Pickup & Return

Who: Rental Staff

Where: Operations > Pickup & Return; Booking/Rental detail

What should show:

• Pickup checklist: identity/license, odometer, fuel, damage, photos, deposit/payment and signature.

• Return checklist: time, odometer, fuel, new damage, late fee, extras, deposit deduction/refund and final total.

What should happen: Pickup makes booking active and vehicle rented. Return calculates final charges and makes vehicle available or sends it to maintenance. Inspection evidence is retained.

```
<b>Backend/API:</b> POST /api/v1/pickups; POST /api/v1/returns; GET /api/v1/rentals/active
```

Main data: rental events, inspections, odometer/fuel records, damage, payments/deposits.

Active Rentals

Who: Manager / Staff

Where: Operations > Active Rentals

What should show:

• Current rentals table/cards: customer, vehicle, pickup time, due time, overdue state and amount due.

• Actions: contact, extend, payment, return.

What should happen: Backend determines overdue state from authoritative rental timestamps. Extension performs fresh availability and pricing checks.

```
<b>Backend/API:</b> GET /api/v1/rentals/active; POST /api/v1/bookings/{id}/extend
```

Main data: bookings, rental state, payments.

Availability

Who: Manager / Staff / Customer

Where: Operations > Availability; Public Vehicle Search

What should show:

• Filters: date, location, category, price and vehicle features.

• Fleet status: available, reserved, rented, maintenance or unavailable.

• Public API exposes only safe published vehicle data.

What should happen: Availability considers vehicle status, overlapping bookings, maintenance blocks and location rules. Database/transaction is authoritative.

```
<b>Backend/API:</b> GET /api/v1/availability; GET /api/v1/public/availability
```

Main data: vehicles, bookings, maintenance blocks, locations, pricing.

Vehicles

Who: Owner / Manager / Fleet Staff

Where: Vehicles > All Vehicles; Vehicle Detail

What should show:

• Table: image, plate, make/model, category, location, status, price, mileage and next service.

• Detail: gallery, specifications, pricing, availability, rental, documents, maintenance, expenses, damage and activity.

• Create/edit: basic → pricing → documents → photos → location → rental settings.

What should happen: Vehicle status transitions are validated against bookings and maintenance. Retirement/unavailability must not silently break active rentals.

```
<b>Backend/API:</b> GET/POST /api/v1/vehicles; GET/PATCH /api/v1/vehicles/{id}; POST /api/v1/vehicles/{id}/status
```

Main data: vehicles, categories, photos, documents, pricing, locations.

Customers

Who: Owner / Manager / Staff

Where: Customers > All Customers; Customer Detail; Verification

What should show:

• Profile, identity/license documents, verification, rental history, payments, deposits, notes and activity.

• Verification queue: pending, approved, rejected, correction required.

• Blacklist with reason and audit history.

What should happen: Customer documents are verified server-side. Blacklist rules can prevent booking according to tenant policy.

```
<b>Backend/API:</b> GET/POST /api/v1/customers; GET/PATCH /api/v1/customers/{id}; POST /api/v1/customers/{id}/verify; POST
/api/v1/customers/{id}/blacklist
```

Main data: customers, documents, verification records, blacklist entries.

Payments

Who: Owner / Finance Staff

Where: Payments > Transactions, Pending, Deposits, Refunds, Invoices

What should show:

• Transaction table: reference, booking, customer, amount, method, provider, status and timestamp.

• Detail: gateway reference, webhook history, refunds and audit events.

What should happen: Payment mutation uses idempotency. Webhook verifies signature and event uniqueness before changing financial state. Refunds create explicit refund records.

```
<b>Backend/API:</b> GET/POST /api/v1/payments; POST /api/v1/payments/{id}/refund; GET /api/v1/deposits; POST
/api/v1/deposits/{id}/refund; POST /api/v1/webhooks/{provider}
```

Main data: payments, payment events, deposits, refunds, invoices.

Contracts

Who: Manager / Staff

Where: Contracts > All, Templates, Signatures, Expiring

What should show:

• Contract detail: customer, booking, terms, signed file, status and timestamps.

• Template editor with variables such as customer, vehicle, dates, price and deposit.

What should happen: Generate contract from booking snapshot. Signature status is tracked. Signed documents are protected. Expiry jobs notify staff.

```
<b>Backend/API:</b> GET/POST /api/v1/contracts; POST /api/v1/contracts/{id}/send; POST /api/v1/contracts/{id}/sign; GET/POST /api/v1/contract-templates
```

Main data: contract templates, contracts, signatures, stored files.

Maintenance

Who: Fleet Staff / Manager

Where: Vehicles > Maintenance; Vehicle Detail

What should show:

• Service queue, schedules, work orders, maintenance history, vendors and costs.

• Vehicle detail shows upcoming and previous service.

What should happen: Issue/due → work order → vehicle may be blocked → service → parts/labor/cost → completion → vehicle can return to available state.

```
<b>Backend/API:</b> GET/POST /api/v1/maintenance; PATCH /api/v1/maintenance/{id}; POST /api/v1/maintenance/{id}/complete
```

Main data: maintenance records, work orders, vendors, costs.

Damage & Inspection

Who: Rental Staff / Manager

Where: Pickup/Return; Vehicle Detail; Rental Detail

What should show:

• Baseline pickup condition; return condition; photos; severity; vehicle location of damage; charge status.

What should happen: Pickup records baseline. Return records differences. Authorized staff can create damage charges linked to invoice/deposit.

```
<b>Backend/API:</b> POST /api/v1/damage; GET /api/v1/vehicles/{id}/damage; POST /api/v1/damage/{id}/charge
```

Main data: damage records, inspection events, photos, financial adjustments.

Expenses

Who: Owner / Finance Staff

Where: Expenses > All, Fuel, Maintenance Costs, Other, Categories

What should show:

• Table: date, category, vendor, vehicle, amount, method, receipt and notes.

• Vehicle detail can show linked expenses.

What should happen: Create expense → validate category/tenant → attach receipt → optionally link vehicle/vendor → include in reports.

```
<b>Backend/API:</b> GET/POST /api/v1/expenses; PATCH /api/v1/expenses/{id}; GET/POST /api/v1/expense-categories
```

Main data: expenses, categories, receipts, vendors.

Reports

Who: Owner / Manager

Where: Reports > Revenue, Bookings, Fleet Utilization, Customers, Payments, Expenses, P&L

What should show:

• Every report has date range, filters, KPI summary, chart, table and export.

• Large reports should generate asynchronously.

What should happen: Use optimized SQL. Heavy report generation becomes a worker job. Report definitions must use canonical financial and booking rules.

```
<b>Backend/API:</b> GET /api/v1/reports/revenue; GET /api/v1/reports/bookings; GET /api/v1/reports/fleet-utilization; GET /api/v1/reports/pnl
```

Main data: aggregated queries over bookings, payments, vehicles and expenses.

Marketing

Who: Owner / Manager

Where: Marketing > Coupons, Discounts, Campaigns, Referral Codes

What should show:

• Coupon editor: code, discount type, value, dates, limits and restrictions.

• Campaign/referral configuration and redemption tracking.

What should happen: At booking time backend validates eligibility and calculates discount. Never trust a discount amount from the frontend.

```
<b>Backend/API:</b> GET/POST /api/v1/coupons; PATCH/DELETE /api/v1/coupons/{id}; POST /api/v1/coupons/validate
```

Main data: coupons, campaigns, referral codes, redemption records.

Notifications

Who: Manager / System

Where: Notifications > Center, Templates, Automation Rules

What should show:

• Unread/read notification center.

• Templates for email/SMS/WhatsApp/in-app.

• Rules for booking, payment, pickup, return, overdue, document and maintenance events.

What should happen: Business event → queue job → provider → retry/backoff → delivery status. Do not send slow external notifications inside the request handler.

```
<b>Backend/API:</b> GET /api/v1/notifications; PATCH /api/v1/notifications/{id}/read; GET/POST
/api/v1/notification-templates
```

Main data: notifications, templates, delivery attempts, automation rules.

Staff & RBAC

Who: Owner / Admin

Where: Staff > Staff, Roles & Permissions, Activity Logs

What should show:

• Staff list: name, email, role, status, last login.

• Role editor with permission groups.

• Activity log: actor, action, target and timestamp.

What should happen: Invite → membership pending → acceptance → role assignment. Sensitive changes create audit logs. Backend enforces every permission.

```
<b>Backend/API:</b> GET/POST /api/v1/users; GET/POST /api/v1/roles; PATCH /api/v1/users/{id}/roles; GET /api/v1/audit-logs
```

Main data: users, tenant memberships, roles, permissions, audit logs.

Online Booking

Who: Customer / Tenant Owner

Where: Public tenant website; Online Booking settings

What should show:

• Pages: Home, Search, Vehicle Details, Checkout, Confirmation, Account, Upcoming Bookings, History, Profile.

• Tenant controls branding, pricing visibility, rules and booking settings.

What should happen: Search → availability → vehicle → customer → documents if required → pricing → payment → booking → confirmation. Uses same backend availability/pricing rules as staff.

```
<b>Backend/API:</b> GET /api/v1/public/config; GET /api/v1/public/vehicles; GET /api/v1/public/availability; POST
/api/v1/public/bookings
```

Main data: published vehicles, public settings, bookings, customers, payments.

Subscription

Who: Tenant Owner / Platform Admin

Where: Subscription > Plan, Billing, Usage, Invoices

What should show:

• Current plan, limits, usage, renewal, invoices and payment state.

What should happen: Plan changes create subscription events. Limits are enforced server-side; usage counters must be safe under concurrent requests.

```
<b>Backend/API:</b> GET /api/v1/subscription; POST /api/v1/subscription/change; GET /api/v1/subscription/usage; GET
/api/v1/subscription/invoices
```

Main data: plans, subscriptions, events, invoices, usage.

Settings

Who: Tenant Owner / Admin

Where: Settings

What should show:

• Company; Locations; Business Hours; Rental Rules; Pricing; Tax; Payment Methods; Integrations; Branding; Security.

What should happen: All settings are tenant-scoped. Rental rules feed booking validation. Pricing feeds the canonical pricing service. Sensitive changes are audited.

```
<b>Backend/API:</b> GET/PATCH /api/v1/settings/company; GET/PATCH /api/v1/settings/rental-rules; GET/PATCH
/api/v1/settings/pricing; GET/PATCH /api/v1/settings/branding
```

Main data: tenant settings, locations, business hours, pricing/tax rules, integrations.

4. End-to-end workflows

Customer booking → rental: Public Search → Availability → Vehicle Details → Pricing → Customer/Identity → Payment → Booking → Confirmation → Pickup Queue → Pickup Inspection → Active Rental → Return Inspection → Final Charges → Deposit Refund/Deduction → Invoice → Vehicle Available/Maintenance.

Staff booking → rental: Customer → Dates/Location → Availability → Pricing → Confirmation → Payment/Deposit → Contract → Pickup → Active Rental → Return → Invoice.

Maintenance: Service due/issue → Work Order → Vehicle blocked when required → Service → Cost → Inspection → Complete → Available/other valid status.

Payment webhook: Gateway → Signature verification → Idempotency check → Transaction update → Booking financial state → Audit → Notification job.

Overdue rental: Worker detects overdue → event → policy-driven late fee → staff/customer notification → audit.

Subscription limit: Request → authenticated tenant → plan/usage check → allow or standardized limit error → safely update usage.

5. Source-of-truth rules

• Availability: backend/database is authoritative.

• Pricing: backend pricing service is authoritative; frontend calculations are previews.

• Payment status: verified provider/backend state is authoritative.

• Rental/booking status: backend state machine is authoritative.

• Permissions: backend authorization is authoritative.

• Tenant identity: authenticated membership/context is authoritative.

• Reports: backend query definitions are authoritative.

6. UI placement rules

• Use full pages for complex workflows and primary data management.

• Use Drawers for quick create/edit/detail/filter interactions.

• Use Modals for destructive confirmations and short single-step actions.

• Use Tables for operational lists and filtering.

• Use Cards for KPIs and compact entity previews.

• Use Timelines for activity/history.

• Use public full-page flows for customer booking and checkout.

7. Anti-vibe implementation rules

• Inspect the existing repository before coding.

• Define domain model, API contract, permissions and state transitions before implementation.

• Implement one coherent vertical slice at a time.

• Handlers stay thin; services own business rules; SQLC/repositories own database access.

• Never trust client-supplied tenant IDs, prices, payment success, permissions or availability.

• Do not create fake endpoints, fake success responses, dead buttons or placeholder features presented as complete.

• Do not duplicate pricing, availability, authorization or financial rules.

• Every critical rule gets automated tests.

• Verify tenant isolation, transactions, idempotency and error states before marking a feature complete.

8. Feature Definition of Done

• Clear purpose, user and UI location.

• API request/response contract documented.

• Database schema/indexes/constraints defined.

• Authentication and authorization enforced.

• Tenant isolation enforced.

- Loading, empty, validation and error states handled in frontend.
- Business rules implemented server-side and tested.
- Sensitive mutations audited.
- Background jobs are retry-safe and idempotent.
- No dead buttons or fake functionality.
- Go tests, integration tests and migration checks pass.